

CAHIER DES CHARGES

Plateforme E-commerce Cosmétique de Luxe

Site Web + Panel d'Administration + API Backend

Stack technique : Next.js · NestJS · PostgreSQL · Docker

Paielement en ligne : Stripe | Livraison : Amana

Version 1.0

Août 2026

Document confidentiel — usage interne / prestataire

Sommaire

Sommaire	2
1. Présentation du projet	4
1.1 Contexte.....	4
1.2 Objectifs du projet.....	4
1.3 Public cible.....	4
1.4 Périmètre du projet.....	4
2. Architecture technique globale.....	5
2.1 Stack technologique retenue.....	5
2.2 Schéma d'architecture (vue logique).....	5
2.3 Environnements.....	6
3. Spécifications fonctionnelles — Site public (Front Office)	7
3.1 Page d'accueil	7
3.2 Catalogue produits	7
3.3 Fiche produit.....	7
3.4 Panier.....	7
3.5 Tunnel de commande (Checkout).....	7
3.6 Paiement en ligne — Stripe.....	8
3.7 Livraison — Amana.....	8
3.8 Espace client (mon compte)	8
3.9 Contenu & autres pages.....	9
3.10 Expérience & responsive.....	9
4. Spécifications fonctionnelles — Panel d'administration	10
4.1 Tableau de bord (Dashboard).....	10
4.2 Gestion des produits.....	10
4.3 Gestion des stocks.....	10
4.4 Gestion des commandes.....	10
4.5 Gestion des clients	10
4.6 Marketing & promotions.....	10
4.7 Gestion du contenu (CMS léger).....	11
4.8 Livraison & expédition	11
4.9 Utilisateurs admin & rôles (RBAC).....	11
4.10 Statistiques & rapports	11
4.11 Paramètres généraux.....	11
5. Modèle de données (PostgreSQL)	12
5.1 Bonnes pratiques base de données	12

6. Intégrations tierces	14
6.1 Stripe (paiement)	14
6.2 Amana (livraison)	14
6.3 Emailing transactionnel.....	14
6.4 Autres intégrations possibles	14
7. Sécurité de la plateforme	15
7.1 Authentification & gestion des accès	15
7.2 Protection de l'application (OWASP Top 10).....	15
7.3 Sécurité des paiements.....	15
7.4 Sécurité réseau & infrastructure.....	15
7.5 Protection des données personnelles	16
7.6 Journalisation, sauvegarde & supervision.....	16
7.7 Sécurité organisationnelle.....	16
8. Conteneurisation & déploiement (Docker)	17
8.1 Services Docker Compose	17
8.2 Bonnes pratiques Docker	17
9. Livrables attendus.....	18
10. Planning indicatif (phases)	19
11. Maintenance & évolutivité	20

1. Présentation du projet

1.1 Contexte

Le projet consiste à concevoir et développer une plateforme e-commerce complète dédiée à la vente de produits cosmétiques haut de gamme (soins visage, sérums, crèmes, produits capillaires, parfums, coffrets), sur le modèle d'un site tel que kamiracosmetics.ma / kamiracosmetics.com. La plateforme doit couvrir l'intégralité du parcours : découverte des produits, achat en ligne, paiement sécurisé par carte bancaire via Stripe, livraison via Amana (Maroc), ainsi qu'une gestion complète de la boutique via un panel d'administration dédié.

1.2 Objectifs du projet

- Créer un site vitrine + boutique en ligne moderne, rapide, élégant et responsive, à l'image d'une marque de cosmétique premium.
- Permettre la gestion autonome de la boutique (produits, stock, commandes, contenu) via un back-office admin sécurisé.
- Intégrer un système de paiement en ligne fiable et sécurisé (Stripe : carte bancaire, Apple Pay / Google Pay).
- Intégrer la livraison via Amana Express (génération de bordereaux, suivi de colis, paiement à la livraison "Cash on Delivery").
- Garantir un niveau de sécurité élevé (protection des données clients, des paiements, et de l'accès admin).
- Concevoir une architecture technique moderne, scalable et maintenable, conteneurisée avec Docker.
- Support multilingue (Français / Arabe / Anglais) avec gestion RTL pour l'arabe.

1.3 Public cible

- Clientèle B2C, principalement au Maroc et zone MENA, sensible aux produits de beauté haut de gamme.
- Achat depuis mobile (majoritaire), tablette et desktop.
- Équipe interne (gérants de la boutique) utilisant le panel admin au quotidien.

1.4 Périmètre du projet

Le périmètre couvre le développement de 3 grands blocs applicatifs :

- **Front Office (site public)** : vitrine, catalogue, fiche produit, panier, tunnel de commande, compte client, blog, FAQ, contact.
- **Back Office (admin panel)** : gestion produits, stocks, commandes, clients, contenu, promotions, statistiques, rôles/utilisateurs.
- **Backend API + Base de données** : API REST sécurisée (NestJS), base de données relationnelle (PostgreSQL), intégrations tierces (Stripe, Amana, email).

Le tout packagé et orchestré via Docker / Docker Compose pour un déploiement reproductible (environnements dev, staging, production).

2. Architecture technique globale

2.1 Stack technologique retenue

Composant	Technologie	Rôle
Frontend (site public)	Next.js 14+ (React, App Router, TypeScript)	Rendu SSR/SSG pour le SEO, catalogue, tunnel d'achat, compte client
Admin Panel	Next.js (espace protégé /admin) ou app séparée	Interface de gestion back-office pour l'équipe interne
Backend API	NestJS (Node.js, TypeScript)	API REST modulaire, logique métier, sécurité, intégrations
ORM	Prisma ou TypeORM	Mapping objet-relationnel vers PostgreSQL
Base de données	PostgreSQL 16	Stockage relationnel : produits, commandes, clients, stock
Cache / Files d'attente	Redis	Cache, sessions, files d'attente (emails, webhooks)
Stockage fichiers	S3-compatible (AWS S3 / MinIO)	Images produits, factures PDF, exports
Paieement	Stripe API + Webhooks	Paieement carte bancaire, Apple Pay, Google Pay
Livraison	API Amana Express	Création de colis, étiquettes, suivi, COD
Emailing	SendGrid / Resend / SMTP	Confirmation commande, reset mot de passe, notifications
Conteneurisation	Docker + Docker Compose	Environnements reproductibles, déploiement
Reverse proxy	Nginx / Traefik	SSL/TLS, routage, load balancing
CI/CD	GitHub Actions / GitLab CI	Build, tests, déploiement automatisé

2.2 Schéma d'architecture (vue logique)

L'architecture suit un modèle découplé (headless) : le frontend Next.js consomme l'API NestJS via HTTP/REST (JSON), elle-même connectée à PostgreSQL. Les services tiers (Stripe, Amana, Email) sont appelés côté serveur (NestJS) uniquement, jamais exposés côté client, afin de protéger les clés secrètes.

- **Client (navigateur / mobile)** → Next.js (SSR) → NestJS API (REST, JWT) → PostgreSQL
- **NestJS API** → Stripe (paiement) / Amana (livraison) / SMTP (emails) / Redis (cache) / S3 (fichiers)
- **Nginx / Traefik** en frontal : terminaison SSL, redirections HTTPS, sert le frontend et route /api vers NestJS

2.3 Environnements

Environnement	Objectif	Particularités
Développement (dev)	Développement local	Docker Compose local, Stripe en mode test, données fictives
Recette / Staging	Tests fonctionnels avant mise en prod	Copie de la prod, Stripe test, Amana sandbox si disponible
Production	Site live	HTTPS obligatoire, clés API live, sauvegardes automatiques, monitoring

3. Spécifications fonctionnelles — Site public (Front Office)

3.1 Page d'accueil

- Bannière principale (hero) avec mise en avant d'une collection phare (ex. "Gold & Caviar").
- Sections produits mis en avant : nouveautés, meilleures ventes, offres limitées, coffrets/sets.
- Bloc "bénéfices ingrédients" (storytelling de marque, ex. bienfaits or & caviar).
- Bandeau réassurance : livraison, paiement sécurisé, paiement à la livraison, retours.
- Formulaire d'inscription à la newsletter avec incitation (ex. code promo bienvenue).
- Liens réseaux sociaux (Instagram, Facebook, YouTube, TikTok).

3.2 Catalogue produits

- Liste des catégories : Soins visage, Sérums, Crèmes, Contour des yeux, Soins capillaires, Parfums, Coffrets/Sets, Solaires.
- Filtres dynamiques : catégorie, type de peau, prix, contenance, marque/gamme, disponibilité.
- Tri : prix croissant/décroissant, nouveautés, popularité, promotions.
- Pagination ou scroll infini, avec indicateur du nombre de résultats.
- Barre de recherche instantanée (autocomplétion produits).
- Affichage des badges : Nouveau, Promo, Rupture de stock, Bestseller.

3.3 Fiche produit

- Galerie d'images (zoom, plusieurs vues), vidéo produit optionnelle.
- Nom, prix, prix barré si promotion, disponibilité en stock en temps réel.
- Sélecteur de variante (contenance/volume, parfum, teinte) impactant le prix et le stock.
- Description longue, liste d'ingrédients (INCI), mode d'utilisation, précautions.
- Onglets : Description / Composition / Avis clients / Livraison & retours.
- Avis et notation clients (étoiles + commentaires, avec modération admin).
- Produits complémentaires / "vous aimerez aussi" (cross-sell).
- Bouton Ajouter au panier avec sélecteur de quantité, ajout rapide sans quitter la page.
- Ajout à une liste de souhaits (wishlist).
- Partage sur réseaux sociaux.

3.4 Panier

- Mini-panier accessible depuis toutes les pages (icône avec compteur).
- Page panier complète : liste des articles, modification quantité, suppression, sous-total.
- Application de codes promo / coupons de réduction.
- Calcul automatique des frais de livraison selon la zone (ville/région) avant validation.
- Sauvegarde du panier pour utilisateur connecté (persistant entre sessions).
- Estimation du seuil de livraison gratuite ("Plus que X DH pour la livraison offerte").

3.5 Tunnel de commande (Checkout)

- Checkout en plusieurs étapes claires : Informations client → Adresse de livraison → Mode de livraison → Paiement → Confirmation.
- Possibilité de commander en tant qu'invité (guest checkout) ou en se connectant.
- Choix du mode de paiement : carte bancaire (Stripe) ou paiement à la livraison (Cash on Delivery via Amana).
- Validation d'adresse (ville, quartier, code postal) avec liste des villes desservies par Amana.
- Récapitulatif de commande avant validation finale (produits, quantités, frais de port, total TTC).
- Page de confirmation de commande + envoi automatique d'un e-mail récapitulatif.
- Génération automatique de facture / reçu téléchargeable (PDF).

3.6 Paiement en ligne — Stripe

- Intégration Stripe (Stripe Elements / Payment Intents API) pour le paiement par carte bancaire (Visa, Mastercard).
- Support Apple Pay et Google Pay via Stripe (si activé côté compte Stripe).
- Aucune donnée de carte bancaire ne transite ni n'est stockée sur les serveurs de l'application (conformité PCI-DSS déléguée à Stripe).
- Gestion des Webhooks Stripe côté NestJS pour confirmer le paiement de façon asynchrone et fiable (paiement réussi, échoué, remboursement).
- Gestion des remboursements (total ou partiel) déclenchable depuis le panel admin.
- Page dédiée en cas d'échec de paiement avec possibilité de réessayer.

3.7 Livraison — Amana

- Intégration à l'API Amana Express (ou Amana Distribution) pour la création automatique de colis à la validation d'une commande payée ou confirmée.
- Génération automatique du bordereau d'expédition (étiquette) et du numéro de suivi (tracking number).
- Support du paiement à la livraison (Cash on Delivery / COD), avec collecte du montant par Amana et réconciliation en admin.
- Suivi de commande côté client : statut en temps réel (préparée, expédiée, en cours de livraison, livrée, retournée) via webhook ou synchronisation périodique avec Amana.
- Calcul des frais de livraison selon zone géographique (grille tarifaire configurable en admin : par ville/région, poids ou forfait).
- Gestion des zones non desservies / hors zone de livraison.

3.8 Espace client (mon compte)

- Création de compte / connexion (email + mot de passe, option connexion via Google).
- Historique des commandes avec statut et suivi de livraison.
- Gestion des adresses de livraison (ajout, modification, adresse par défaut).
- Gestion des informations personnelles et du mot de passe.
- Liste de souhaits (wishlist).
- Téléchargement des factures.
- Suppression de compte / demande d'export de données personnelles (conformité RGPD / loi 09-08).

3.9 Contenu & autres pages

- Page "À propos" (histoire de la marque, valeurs).
- Blog (articles beauté, conseils) géré depuis l'admin, avec catégories et SEO par article.
- Page FAQ (questions fréquentes, gérée dynamiquement depuis l'admin).
- Page Contact (formulaire + coordonnées + carte).
- Pages légales : Conditions Générales de Vente, Politique de confidentialité, Politique de retour/remboursement, Mentions légales.
- Multilingue FR / AR (RTL) / EN avec sélecteur de langue.
- Multi-devise potentielle (MAD par défaut, EUR/USD pour la version .com internationale).

3.10 Expérience & responsive

- Design responsive mobile-first (mobile, tablette, desktop).
- Temps de chargement optimisé (images compressées/optimisées, lazy loading, SSR/ISR Next.js).
- Optimisation SEO : URLs propres, meta title/description par page et produit, données structurées (Schema.org Product), sitemap.xml, robots.txt.
- Accessibilité de base (contraste, navigation clavier, alt sur images).

4. Spécifications fonctionnelles — Panel d'administration

Le back-office est un espace protégé, accessible uniquement aux utilisateurs autorisés (authentification renforcée), permettant de piloter l'intégralité de la boutique.

4.1 Tableau de bord (Dashboard)

- Vue synthétique : chiffre d'affaires (jour/semaine/mois), nombre de commandes, panier moyen.
- Graphiques de ventes (par période, par catégorie, par produit).
- Alertes : commandes en attente de traitement, stock faible/rupture, avis en attente de modération.
- Dernières commandes et dernières inscriptions clients.

4.2 Gestion des produits

- CRUD complet des produits : création, modification, suppression, duplication.
- Gestion des variantes (contenance, teinte, parfum) avec prix et stock indépendants.
- Upload multiple d'images (drag & drop), réordonnancement, image principale.
- Gestion des catégories et sous-catégories (arborescence).
- Gestion du SEO par produit (titre, description, slug/URL).
- Statut produit : brouillon, publié, archivé, en rupture.
- Import / export en masse (CSV/Excel) pour la gestion catalogue.

4.3 Gestion des stocks

- Suivi du stock en temps réel par produit et par variante.
- Seuils d'alerte de stock faible configurables, avec notification.
- Historique des mouvements de stock (entrées, sorties, ajustements manuels).
- Blocage automatique de l'achat si rupture de stock.

4.4 Gestion des commandes

- Liste des commandes avec filtres (statut, date, mode de paiement, mode de livraison).
- Détail commande : produits, client, adresse, paiement, livraison, historique des statuts.
- Changement de statut (nouvelle, confirmée, en préparation, expédiée, livrée, annulée, remboursée).
- Génération / téléchargement du bordereau Amana et de la facture directement depuis la commande.
- Gestion des remboursements Stripe (total/partiel) depuis la fiche commande.
- Export des commandes (CSV/Excel/PDF) pour comptabilité.

4.5 Gestion des clients

- Liste des clients avec historique d'achats et valeur totale (LTV).
- Fiche client détaillée : coordonnées, adresses, commandes, notes internes.
- Possibilité de bloquer un compte client (fraude, abus).

4.6 Marketing & promotions

- Création de coupons de réduction (pourcentage, montant fixe, livraison offerte), avec conditions (date de validité, montant minimum, produits/catégories concernés, usage unique ou multiple).
- Gestion des ventes flash / promotions temporaires.
- Gestion de la newsletter (export des abonnés, intégration avec outil d'emailing).
- Gestion des avis clients (modération, réponse, mise en avant).

4.7 Gestion du contenu (CMS léger)

- Édition des pages statiques (À propos, FAQ, CGV, etc.) via éditeur riche (WYSIWYG).
- Gestion des articles de blog (création, catégories, images, SEO).
- Gestion des bannières et sections mises en avant sur la page d'accueil.
- Gestion des traductions (FR/AR/EN) du contenu éditorial.

4.8 Livraison & expédition

- Configuration des zones de livraison et grille tarifaire (par ville/région).
- Suivi centralisé des expéditions Amana : statut, numéro de suivi, historique.
- Réconciliation des paiements Cash on Delivery (COD) collectés par Amana.

4.9 Utilisateurs admin & rôles (RBAC)

- Gestion multi-utilisateurs du back-office avec rôles : Super Admin, Gestionnaire produits, Gestionnaire commandes, Support client, Comptabilité.
- Permissions granulaires par module (lecture seule / lecture-écriture / suppression).
- Authentification à deux facteurs (2FA) obligatoire pour les comptes Super Admin.
- Journal d'audit (audit log) : traçabilité de toutes les actions sensibles (qui a fait quoi et quand).

4.10 Statistiques & rapports

- Rapports de ventes par période, par produit, par catégorie.
- Taux de conversion, panier moyen, produits les plus/moins vendus.
- Rapport sur les abandons de panier (relance possible par email).
- Export des données pour analyse externe (Excel/CSV).

4.11 Paramètres généraux

- Informations de l'entreprise (nom, adresse, ICE, coordonnées bancaires).
- Paramétrage des clés API (Stripe, Amana, SMTP) via variables d'environnement sécurisées — jamais en dur dans le code.
- Paramétrage des taxes (TVA), devises, langues actives.
- Gestion des modèles d'e-mails transactionnels (confirmation commande, expédition, etc.).

5. Modèle de données (PostgreSQL)

Base de données relationnelle PostgreSQL, accédée via un ORM (Prisma ou TypeORM) depuis NestJS. Ci-dessous les entités principales du modèle conceptuel de données (MCD) — la liste n'est pas exhaustive et sera affinée en phase de conception détaillée.

Table	Description sommaire
users	Comptes clients : email, mot de passe (haché), nom, téléphone, rôle, statut, dates.
admin_users	Comptes back-office : email, mot de passe haché, rôle, 2FA, dernière connexion.
roles / permissions	Rôles admin et permissions associées (RBAC).
addresses	Adresses de livraison/facturation liées à un utilisateur.
categories	Catégories/sous-catégories produits (arborescence).
products	Produits : nom, description, prix, statut, SEO, catégorie.
product_variants	Variantes (contenance, teinte...), prix, SKU, stock.
product_images	Images liées à un produit, ordre d'affichage.
stock_movements	Historique des mouvements de stock (entrée/sortie/ajustement).
carts / cart_items	Panier et lignes de panier, liés à un utilisateur ou une session invité.
orders	Commandes : statut, totaux, adresse, mode de livraison/paiement, dates.
order_items	Lignes de commande : produit, variante, quantité, prix unitaire.
payments	Paiements liés à une commande : Stripe payment_intent_id, statut, montant.
shipments	Expéditions Amana : tracking number, statut, bordereau, COD collecté.
coupons	Codes promo : type, valeur, conditions, dates de validité, usage.
reviews	Avis clients : note, commentaire, statut de modération, produit lié.
wishlists	Produits favoris par utilisateur.
blog_posts	Articles de blog : contenu, catégorie, SEO, traductions.
pages	Pages statiques (CGV, FAQ...) éditables, multilingues.
newsletter_subscribers	Abonnés newsletter.
audit_logs	Journal des actions effectuées par les administrateurs.

5.1 Bonnes pratiques base de données

- Clés primaires en UUID plutôt qu'en entiers auto-incrémentés (évite l'énumération d'IDs).
- Contraintes d'intégrité référentielle (clés étrangères) et contraintes NOT NULL/UNIQUE strictes.
- Index sur les colonnes de recherche/filtrage fréquentes (slug produit, email, statut commande).
- Champs monétaires stockés en entiers (centimes) pour éviter les erreurs d'arrondi flottant.

- Horodatage systématique (created_at, updated_at) et soft-delete (deleted_at) pour les entités critiques.
- Migrations versionnées (Prisma Migrate / TypeORM migrations), jamais de modification manuelle en production.

6. Intégrations tierces

6.1 Stripe (paiement)

- Utilisation de l'API Stripe Payment Intents pour un paiement sécurisé côté serveur (NestJS), clé secrète jamais exposée côté client.
- Utilisation de Stripe.js / Stripe Elements côté frontend pour la saisie sécurisée de la carte (aucune donnée carte ne transite par le backend applicatif).
- Webhook Stripe dédié (endpoint signé et vérifié) pour recevoir les événements : `payment_intent.succeeded`, `payment_intent.payment_failed`, `charge.refunded`.
- Mode test / mode live clairement séparés selon l'environnement (variables d'environnement distinctes).

6.2 Amana (livraison)

- Intégration API pour la création de colis (expéditeur, destinataire, poids, valeur, mode de paiement COD ou prépayé).
- Récupération du numéro de suivi et du bordereau PDF pour impression en entrepôt.
- Mise à jour du statut de livraison (webhook si disponible, sinon synchronisation périodique planifiée).
- Gestion des cas d'échec de livraison / retour (NPAI, refus client).

6.3 Emailing transactionnel

- Envoi automatique : confirmation de commande, confirmation de paiement, expédition/tracking, réinitialisation de mot de passe, relance panier abandonné.
- Fournisseur recommandé : SendGrid, Resend ou Amazon SES, via file d'attente (Redis/Bull) pour fiabilité et retry automatique.

6.4 Autres intégrations possibles

- Google Analytics 4 / Meta Pixel pour le suivi marketing (avec bandeau de consentement cookies).
- Google reCAPTCHA / hCaptcha sur les formulaires publics (inscription, contact, avis).
- Réseaux sociaux : partage produit, connexion sociale (Google Login) optionnelle.

7. Sécurité de la plateforme

La sécurité est traitée comme une exigence transversale, appliquée à tous les niveaux : réseau, application, données, et organisation.

7.1 Authentification & gestion des accès

- Authentification par JSON Web Token (JWT) : access token de courte durée + refresh token stocké en cookie HttpOnly/Secure.
- Mots de passe hachés avec bcrypt ou argon2 (jamais stockés en clair, jamais réversibles).
- Politique de mot de passe robuste (longueur minimale, complexité) côté inscription client et admin.
- Authentification à deux facteurs (2FA / TOTP) obligatoire pour les comptes administrateurs.
- Contrôle d'accès basé sur les rôles (RBAC) avec vérification systématique côté backend (jamais uniquement côté frontend).
- Verrouillage temporaire du compte après plusieurs tentatives de connexion échouées (protection brute-force).
- Expiration de session automatique et déconnexion sur inactivité prolongée pour l'admin.

7.2 Protection de l'application (OWASP Top 10)

- Validation stricte de toutes les entrées utilisateur côté backend (DTO + class-validator sous NestJS), rejet des données non conformes.
- Protection contre les injections SQL via l'utilisation exclusive de l'ORM avec requêtes paramétrées (aucune requête SQL brute concaténée).
- Protection XSS : échappement systématique des données affichées, Content-Security-Policy stricte, sanitisation du contenu riche (CMS/blog).
- Protection CSRF sur les actions sensibles (tokens CSRF ou SameSite cookies + vérification d'origine).
- Limitation du taux de requêtes (rate limiting) sur les endpoints sensibles : login, réinitialisation mot de passe, paiement, API publique.
- Protection contre les attaques par force brute et le credential stuffing (throttling + captcha après échecs répétés).
- En-têtes de sécurité HTTP : Strict-Transport-Security (HSTS), X-Content-Type-Options, X-Frame-Options, Content-Security-Policy, Referrer-Policy.
- Vérification systématique de la signature des webhooks entrants (Stripe, Amana) pour empêcher les faux appels.

7.3 Sécurité des paiements

- Aucune donnée de carte bancaire n'est jamais stockée ni ne transite par les serveurs applicatifs (délégation totale à Stripe, conformité PCI-DSS niveau SAQ A).
- Toutes les communications avec Stripe s'effectuent en HTTPS via les SDK officiels.
- Idempotence des appels de paiement pour éviter les doubles débits en cas de re-tentative réseau.

7.4 Sécurité réseau & infrastructure

- HTTPS obligatoire sur l'ensemble du site (certificat TLS via Let's Encrypt ou équivalent), redirection automatique HTTP → HTTPS.

- Reverse proxy (Nginx/Traefik) en frontal, backend et base de données non exposés directement à Internet.
- Base de données accessible uniquement depuis le réseau interne Docker (aucun port PostgreSQL public).
- Segmentation des conteneurs Docker par réseau interne dédié, principe du moindre privilège entre services.
- Secrets (clés API Stripe/Amana, identifiants DB, JWT secret) gérés via variables d'environnement / gestionnaire de secrets, jamais commités dans le code.
- Pare-feu applicatif (WAF) et protection anti-DDoS de base au niveau du reverse proxy / CDN.

7.5 Protection des données personnelles

- Conformité avec la loi marocaine 09-08 relative à la protection des données personnelles (et RGPD pour la clientèle européenne du .com).
- Politique de confidentialité claire et accessible, consentement explicite pour les cookies non essentiels.
- Droit d'accès, de rectification et de suppression des données personnelles, exerçable depuis l'espace client ou sur demande.
- Chiffrement des données sensibles au repos (disques chiffrés) et en transit (TLS).
- Minimisation des données collectées : uniquement les informations nécessaires au traitement des commandes.

7.6 Journalisation, sauvegarde & supervision

- Journal d'audit (audit log) de toutes les actions administrateur sensibles (création/suppression produit, remboursement, changement de rôle).
- Sauvegardes automatiques et régulières de la base de données (quotidiennes), avec rétention et tests de restauration périodiques.
- Monitoring applicatif et infrastructure (uptime, erreurs 5xx, temps de réponse) avec alertes.
- Centralisation des logs applicatifs, sans jamais logger de données sensibles (mots de passe, numéros de carte).
- Plan de reprise d'activité (PRA) documenté en cas d'incident majeur.

7.7 Sécurité organisationnelle

- Séparation stricte des environnements (dev/staging/prod) avec des identifiants et clés API distincts.
- Revue de code obligatoire avant fusion (pull request) sur les branches principales.
- Analyse automatisée des dépendances (audit npm) pour détecter les vulnérabilités connues.
- Tests de sécurité (a minima un test d'intrusion / audit de sécurité avant la mise en production).

8. Conteneurisation & déploiement (Docker)

8.1 Services Docker Compose

Service	Image / Base	Rôle
frontend	node:20-alpine (Next.js build)	Site public + espace client
admin	node:20-alpine (Next.js build)	Panel d'administration (ou route protégée du frontend)
backend-api	node:20-alpine (NestJS)	API REST, logique métier, intégrations
postgres	postgres:16-alpine	Base de données principale
redis	redis:7-alpine	Cache, sessions, files d'attente
nginx / traefik	nginx:alpine ou traefik	Reverse proxy, SSL/TLS, routage
worker (optionnel)	node:20-alpine	Traitement asynchrone (emails, synchronisation Amana)
minio (optionnel)	minio/minio	Stockage objet compatible S3 pour les images

8.2 Bonnes pratiques Docker

- Un Dockerfile multi-stage par service (build puis image finale légère) pour réduire la taille des images.
- Fichiers docker-compose.yml distincts pour dev (avec hot-reload) et production (images optimisées).
- Variables d'environnement injectées via fichiers .env non versionnés (exclus via .gitignore), avec .env.example fourni.
- Volumes persistants dédiés pour PostgreSQL et les fichiers uploadés, avec stratégie de sauvegarde des volumes.
- Healthchecks Docker configurés sur chaque service critique (API, DB) pour un redémarrage automatique en cas de défaillance.
- Conteneurs exécutés avec un utilisateur non-root autant que possible.
- Pipeline CI/CD : build automatique des images, tests, puis déploiement (blue/green ou rolling update) en production.

9. Livrables attendus

- Code source complet (frontend, admin, backend API) versionné sur dépôt Git, avec documentation technique.
- Fichiers Docker / Docker Compose prêts à l'emploi pour les environnements dev et production.
- Documentation de l'API (Swagger/OpenAPI généré depuis NestJS).
- Documentation d'installation et de déploiement.
- Panel d'administration fonctionnel avec comptes de démonstration.
- Jeu de données de test (produits, catégories, utilisateurs) pour la recette.
- Guide d'utilisation du back-office à destination de l'équipe métier.

10. Planning indicatif (phases)

Phase	Contenu	Durée indicative
1. Cadrage & maquettage	Validation du cahier des charges, maquettes UI/UX (desktop + mobile)	2 semaines
2. Mise en place infrastructure	Setup Docker, CI/CD, environnements dev/staging	1 semaine
3. Développement backend	API NestJS, base de données, authentification, modules métier	4 à 6 semaines
4. Développement frontend	Site public Next.js (catalogue, panier, checkout, compte client)	4 à 6 semaines
5. Développement admin panel	Back-office complet (produits, commandes, stats, rôles)	3 à 4 semaines
6. Intégrations tierces	Stripe, Amana, emailing, webhooks	2 à 3 semaines
7. Sécurisation & audit	Durcissement sécurité, tests, audit avant mise en ligne	1 à 2 semaines
8. Recette & tests	Tests fonctionnels, tests de charge, corrections	2 semaines
9. Mise en production	Déploiement, DNS, SSL, formation équipe	1 semaine

Ce planning est indicatif et sera affiné selon les ressources allouées au projet et les retours de validation à chaque phase.

11. Maintenance & évolutivité

- Mises à jour de sécurité régulières des dépendances (frontend, backend, images Docker).
- Support technique post-livraison (corrective, évolutive) selon accord contractuel séparé.
- Architecture pensée pour évoluer : ajout futur d'une application mobile consommant la même API, marketplace, programme de fidélité, multi-boutiques.
- Supervision continue des performances et de la disponibilité du site.

Fin du cahier des charges.